My feature: Shelter and First-Night Survival.

Teammates' features: Abram – Model behavior , Kai – Combat, Chayse – LLM to Baritone command layer

1 . The problem, who has it, and the business-level goal in one or two sentences. Then two lists: In scope for this semester, and Out of scope. Out of scope is not optional, because it is where an agent's invented features show up.

Actionable is a minecraft mod that extends on the pre existing library baritone. This mod is used for an LLM to take full control of your minecraft player and progress through the game for you. Many users and researchers can use this to test the behavior of LLMs in a very popular game. In scope for this project is what the LLM is capable of doing currently such as cutting down trees, building a house, finding sheep for a bed, getting food, mining for coal, iron, and stone , and most basic early game survival strategies. Where the project may be out of scope is late game survival such as a exploring in large caves or finishing the entire game, multiplayer modes, things like making a nether portal and knowing where to go in the nether to further progress.

2. Stakeholders and users Every role that uses or cares about the system (player, admin, family member, the grader, the device it runs on), and what each one needs from it.

(U-1) Minecraft players would be able to start, pause, stop, and redirect autonomous play while understanding what the bot is doing. (U-2)Players using autonomous survival would have the bot gather resources, build shelter, obtain food, and respond to danger without repeatedly attempting completed tasks. (U-3)User installing/configuring the mod is provided clear installation instructions, compatible dependencies, and understandable errors when the configured model is unavailable. (U-4) Development team would have access to logs and automated tests that help reproduce failures and maintain planning, crafting, navigation, and survival behavior. (U-5) Instructor/grader is given clearly stated requirements and observable checks that demonstrate whether the project meets them.

3. Context, constraints and assumptions

The choices already made for you: language, platform, build system, hardware, libraries, licences, deadline, team size. Then the assumptions you are making and the

dependencies you rely on. Mark each assumption so a teammate can confirm or correct it.

Actionable is a client-side Minecraft 26.3 mod written in Java 25, built with Gradle and Fabric Loom, and tested using JUnit. It depends on Fabric Loader, Fabric API, a compatible Baritone installation, and either Ollama or an authenticated Claude Code CLI for model planning. Initial testing will use private single-player Survival worlds, and users will install compatible dependencies and configure their model provider before running the mod.|

4. Functional requirements

At least 8, numbered FR-1, FR-2, … in a table with these columns (the row is an example):

ID	Requirement	Priority	Traces To	Verified By
FR-1	The system should begin selecting goals when the player joins a world with autonomy enabled.	Must	U-1	Join a world and observe goal selection without entering a command.
FR-2	The system shall replace the current task when the player enters a valid /action <task>	Must	U-1	Start one task, enter another, and verify the active task changes.

	command.			
FR-3	The system shall cancel active Baritone work when the player pauses Actionable.	Must	U-1	Pause during mining and verify mining stops.
FR-4	The system shall validate each model-selected action before execution and reject invalid actions without dispatching them.	Must	U-1, U-3	Supply invalid action fixtures and verify zero actions execute.
FR-5	The system shall select gathering 20 logs as the first progression objective when starting	Must	U-2	ProgressPlannerTest.emptyInventoryStartsWithWood

	with an empty inventory.			
FR-6	The system shall require every expected wall and roof barrier before declaring the starter house shell complete.	Must	U-2	HouseStructureTest.everySingleWallAndRoofCellIsRequired
FR-7	The system shall plan to obtain missing bed ingredients, craft a bed, and place it inside the house.	Must	U-2	ProgressPlannerTest.bedPlanHuntsSheepCraftsTheBedAndPlacesIt
FR-8	At night without a bed, home-job selection shall skip jobs whose	Must	U-2	HomeJobsTest.jobsThatNeedTheOutsideAreSkippedAtNight

	plans require outdoor gathering.			
FR-9	The system shall reject a model's completion claim when observed inventory quantities or recognized house conditions remain unmet.	Must	U-2	CompletionCheckTest.insufficientItemsAndMissingHouseOverrideModelClaims
FR-10	The system shall save home coordinates and restore them when the same world and dimension are reopened.	Must	U-2, U-4	Save a home, restart the world, and compare restored coordinates.

5. Non-functional requirements

At least 4, numbered NFR-1, ..., in the same table format. Cover at least three different qualities: performance, reliability, usability, accessibility, security, privacy, portability, maintainability. Each

one needs a number, a condition, and a way to measure it. Lecture 4's rewrite of "the app should be fast" is the standard:

ID	Requirement	Priority	Traces to	Verified by
NFR-1	Performance: While the game runs at 20 ticks per second, pausing Actionable shall cancel active Baritone work within 2 client ticks in all 10 trials.	Must	U-1	Record the pause-input tick and cancellation tick during 10 mining or movement trials; require each difference to be ≤2 ticks.
NFR-2	Reliability: When a planner response arrives after its request was invalidated by pausing, the system shall execute zero actions from that response across 10 trials.	Must	U-1, U-3	Delay a response, pause, then release it; record dispatched actions in each trial.
NFR-3	Usability: With the debug panel enabled and a task active, the display shall show the current goal, current step, and latest action result in all 5 prepared task scenarios.	Should	U-1	Capture the panel during each scenario and check that all three fields appear.
NFR-4	Maintainability: Every change to a decision rule that runs independently of Minecraft shall include at least 1 automated regression test, and the Gradle build shall finish with 0 failed tests before merging.	Must	U-4, U-5	Review changed rules and their tests, then inspect the results of ./gradlew build.

6. User Stories

US-1: Control the active task

As a **Minecraft player**, I want to **replace or pause the bot's task** so that **I can control its actions**.

Covers: FR-2, FR-3

- **Given** the bot is gathering wood, **when** I enter a valid `/action get 4 cobblestone` command, **then** the active task changes to gathering cobblestone.
- **Given** the bot is mining in an unwanted location, **when** I pause Actionable, **then** active Baritone mining stops and no further task actions execute while paused.

US-2: Build a complete shelter

As a **Minecraft player**, I want **the bot to build and check its shelter** so that **it does not mistake an unfinished house for a completed one**.

Covers: FR-5, FR-6, FR-9

- **Given** an empty inventory and no completed foundation objectives, **when** autonomous progression selects its first objective, **then** it selects gathering 20 logs.
- **Given** a house with one missing roof barrier, **when** the model claims the house is complete, **then** the system keeps the house-shell objective incomplete.

US-3: Prepare for the night

As a **Minecraft player**, I want **the bot to prepare a bed and choose suitable nighttime jobs** so that **it can continue early survival progression**.

Covers: FR-7, FR-8

- **Given** a completed house, a crafting table, and sufficient bed ingredients, **when** the bed objective is selected, **then** the plan includes crafting and placing a bed inside the house.
- **Given** nighttime, no bed, and insufficient materials for a job that requires outdoor gathering, **when** home-job selection runs, **then** it skips that job.

US-4: Validate model actions

As a **player using a model provider**, I want **model actions checked before execution** so that **invalid responses do not cause unintended commands**.

Covers: FR-4

- **Given** a supported action with valid arguments, **when** validation succeeds, **then** the action is accepted for execution.
- **Given** a model response containing an unsupported action type or malformed arguments, **when** validation runs, **then** the action is rejected and no game command is dispatched from it

US-5: Restore the saved home
As a Minecraft player, I want the bot to remember its home across restarts so that it can continue using the same location.
Covers: FR-10
- Given saved home coordinates for the current world and dimension, when I restart and rejoin that world, then the system restores the same home coordinates.
- Given a missing or malformed home file, when the system attempts to load it, then the home remains unknown and the client continues running; a malformed file produces a warning.
US-6: Verify changes to decision rules
As a developer, I want automated regression tests for changes to decision rules so that I can detect broken behavior before merging changes.
Covers: NFR-4
- Given a behavior change to a rule that runs independently of Minecraft, when the change is reviewed, then it includes at least one automated regression test and a Gradle build reporting zero failed tests.
- Given a regression test fails, when the Gradle build runs, then the build reports failure and the change does not meet the merge acceptance criteria.

7. Traceability Matrix

Goal (G-1): Enable an LLM to autonomously complete early-game Minecraft survival in a single-player world, with the player able to observe and control it.

Goal	User need	Requirement IDs	User stories	How it is verified
G-1	U-1: Control and understand autonomous play	FR-1, FR-2, FR-3; NFR-1, NFR-3	US-1	Join-world startup check; task replacement check; 10 pause-timing trials; debug-panel review across 5 scenarios.
G-1	U-2: Gather resources and establish a	FR-5, FR-6, FR-9	US-2	Empty-inventory progression test; missing-wall/roof tests; premature-completion rejection tests.

	usable shelter			
G-1	U-2: Prepare for nighttime survival	FR-7, FR-8	US-3	Bed crafting/placement plan test; nighttime outdoor-job exclusion test.
G-1	U-3: Use the configured model safely and understand failures	FR-4; NFR-2	US-4	Invalid-action validation tests; 10 delayed-response trials after pause.
G-1	U-2, U-4: Preserve home progress across restarts	FR-10	US-5	Save home coordinates, restart the same world/dimension, and compare restored coordinates.
G-1	U-4: Maintain and diagnose the system	NFR-4	US-6	Review regression-test coverage for changed rules; run `./gradlew build` and require zero failed tests.
G-1	U-5: Evaluate the project using observable evidence	NFR-4	US-6	Review requirement-to-test links and the recorded build/test report.

8. Open Questions

Open question	Who can answer it?	Assumption in the meantime
Does my shelter and first-night survival feature overlap with another teammate's chosen feature?	All teammates	My proposed feature includes the starter house, bed preparation, and nighttime home jobs until ownership is confirmed.
Is the semester's final objective reliable early survival or successfully entering the Nether?	Team and instructor	Early survival is the main acceptance target; Nether entry is a broader progression objective.
Which computer, world seed, and difficulty should we use for acceptance testing?	Team members responsible for testing	Use a private single-player Survival world and record the machine specifications, seed, and difficulty with each test.
What should happen when saved home data is corrupted: create a new home, ask the player, or attempt recovery?	Team and users	Leave the home unknown, log a warning, and continue running without promising automatic recovery.
What is the exact starter-house blueprint?	Team	The agent's layout is temporary until the team confirms one.
Which blocks count as a valid wall or roof?	Me (Shelter owner)	Only full, solid, dry blocks count.
What should the bot do when sheep or other required resources remain unavailable?	Progression-feature owner and team	Keep the objective incomplete and use the existing retry/replanning behavior; do not promise completion within a fixed time.

Does a partly built house prove the 20 logs were gathered?	Progression owner	No, a partial house is not proof.
What counts as “night”: a time window, sleep eligibility, or weather?	Team	Weather alone does not count as night.
How often should the bot retry sleeping, and how should refusals be handled?	Team	A refused sleep never counts as success.

Part 2. Your feature, in depth:

Feature: Shelter and First-Night Survival

Use case: Establish a usable starter home

Actor: Minecraft player using Actionable.

Preconditions: The player has joined a Survival world, Actionable is enabled, and its required dependencies are available. The main success path assumes a suitable building site and reachable starter materials.

Trigger: Autonomous progression detects that the starter house or bed is incomplete.

Main flow:

1. The system reads the player’s inventory, saved home location, time of day, and loaded house conditions.
2. The system selects the earliest unmet foundation objective, beginning with gathering 20 logs when the inventory is empty.
3. The system obtains required materials and builds the fixed starter-house shell.

4. The system checks every required wall and roof barrier, prepares a dry supporting floor, and clears the interior and entrance.
5. The system installs a crafting table and a complete wooden door.
6. The system obtains missing bed ingredients, crafts a bed, and places it inside the house.
7. The system checks the house and bed against observed world conditions and marks only satisfied objectives complete.
8. If it is night and Minecraft permits sleeping, the system interacts with the bed to attempt sleep.

Alternate flows:

- **1a — Saved home data is malformed:** The system logs a warning, leaves the home location unknown, and continues through the normal no-home path.
- **3a — Required materials cannot be obtained:** The system keeps the objective incomplete and reports the failure to the retry/replanning process.
- **4a — A wall or roof barrier is missing:** The system keeps the shell incomplete and selects repair work before declaring the house complete.
- **6a — A bed is already held:** The system places the existing bed instead of gathering replacement wool.
- **6b — Night begins before a bed is available:** The system selects a feasible home job that requires no outdoor gathering. If no job is feasible, it waits indoors and reevaluates when day returns.
- **8a — Minecraft refuses sleep:** The system does not record successful sleep. The handling of the refusal reason remains an open question for the team.

Postcondition: On successful completion, the loaded house passes its enclosure checks and a bed is observed inside it:
house.enclosed == true AND bed.placedInsideHouse == true

Mermaid Diagram:



Part 3. The agent check

Appendix
Appendix: Agent check

Tool and model: Claude Opus 5.5

Exact prompt:
Prompt:

Implement the shelter and first-night survival feature described below.

You do not have access to my repository. Base your implementation

only on these user stories and this use case. Provide code and explain

your implementation choices.

USER STORIES:

6. User Stories

US-1: Control the active task

As a Minecraft player, I want to replace or pause the bot's task so that I can control its actions.

Covers: FR-2, FR-3

Given the bot is gathering wood, when I enter a valid /action get 4 cobblestone command, then the active task changes to gathering cobblestone.

Given the bot is mining in an unwanted location, when I pause Actionable, then active Baritone mining stops and no further task actions execute while paused.

US-2: Build a complete shelter

As a Minecraft player, I want the bot to build and check its shelter so that it does not mistake an unfinished house for a completed one.

Covers: FR-5, FR-6, FR-9

Given an empty inventory and no completed foundation objectives, when autonomous progression selects its first objective, then it selects gathering 20 logs.

Given a house with one missing roof barrier, when the model claims the house is complete, then the system keeps the house-shell objective incomplete.

US-3: Prepare for the night

As a Minecraft player, I want the bot to prepare a bed and choose suitable nighttime jobs so that it can continue early survival progression.

Covers:FR-7, FR-8

Given a completed house, a crafting table, and sufficient bed ingredients, when the bed objective is selected, then the plan includes crafting and placing a bed inside the house.

Given nighttime, no bed, and insufficient materials for a job that requires outdoor gathering, when home-job selection runs, then it skips that job.

US-4: Validate model actions

As a player using a model provider, I want model actions checked before execution so that invalid responses do not cause unintended commands.

Covers: FR-4

Given a supported action with valid arguments, when validation succeeds, then the action is accepted for execution.

Given a model response containing an unsupported action type or malformed arguments, when validation runs, then the action is rejected and no game command is dispatched from it

US-5: Restore the saved home

As a Minecraft player, I want the bot to remember its home across restarts so that it can continue using the same location.

Covers: FR-10

- Given saved home coordinates for the current world and dimension, when I restart and rejoin that world, then the system restores the same home coordinates.

- Given a missing or malformed home file, when the system attempts to load it, then the home remains unknown and the client continues running; a malformed file produces a warning.

US-6: Verify changes to decision rules

As a developer, I want automated regression tests for changes to decision rules so that I can detect broken behavior before merging changes.

Covers: NFR-4

- Given a behavior change to a rule that runs independently of Minecraft, when the change is reviewed, then it includes at least one automated regression test and a Gradle build reporting zero failed tests.

- Given a regression test fails, when the Gradle build runs, then the build reports failure and the change does not meet the merge acceptance criteria.

USE CASE:

Feature: Shelter and First-Night Survival

Use case: Establish a usable starter home

Actor: Minecraft player using Actionable.

Preconditions: The player has joined a Survival world, Actionable is enabled, and its required dependencies are available. The main success path assumes a suitable building site and reachable starter materials.

Trigger: Autonomous progression detects that the starter house or bed is incomplete.

Main flow:

The system reads the player's inventory, saved home location, time of day, and loaded house conditions.

The system selects the earliest unmet foundation objective, beginning with gathering 20 logs when the inventory is empty.

The system obtains required materials and builds the fixed starter-house shell.

The system checks every required wall and roof barrier, prepares a dry supporting floor, and clears the interior and entrance.

The system installs a crafting table and a complete wooden door.

The system obtains missing bed ingredients, crafts a bed, and places it inside the house.

The system checks the house and bed against observed world conditions and marks only satisfied objectives complete. If it is night and Minecraft permits sleeping, the system interacts with the bed to attempt sleep.

Alternate flows:

1a — Saved home data is malformed: The system logs a warning, leaves the home location unknown, and continues through the normal no-home path.

3a — Required materials cannot be obtained: The system keeps the objective incomplete and reports the failure to the retry/replanning process.

4a — A wall or roof barrier is missing: The system keeps the shell incomplete and selects repair work before declaring the house complete.

6a — A bed is already held: The system places the existing bed instead of gathering replacement wool.

6b — Night begins before a bed is available: The system selects a feasible home job that requires no outdoor gathering. If no job is feasible, it waits indoors and reevaluates when day returns.

8a — Minecraft refuses sleep: The system does not record successful sleep. The handling of the refusal reason remains an open question for the team.

Postcondition: On successful completion, the loaded house passes its enclosure checks and a bed is observed inside it:

house.enclosed == true AND bed.placedInsideHouse == true

Implementation produced

The agent created a standalone Java module containing shelter progression rules, house inspection, objective tracking, nighttime home-job selection, action validation, and home persistence. It used interfaces in place of Minecraft and Baritone integration. I reviewed its implementation choices to identify details that my specification had not settled.

Decisions and dispositions

1. House geometry: The agent chose three-block-high walls, a flat roof, a front-center door, and fixed furniture positions. Moved to Open questions: The team must confirm the exact starter-house blueprint. These dimensions remain provisional.

2. Acceptable wall materials: The agent excluded glass, slabs, and similar blocks from valid barriers. Moved to Open questions: The shelter owner must determine which block properties qualify as a complete barrier. The interim assumption is a full, solid, dry barrier.

3. Unloaded terrain: The agent reported unloaded house cells as unknown instead of assuming they were complete. Fixed — FR-6 (A): Added an explicit rule that unloaded evidence cannot establish house-shell completion.

4. Initial log milestone: The agent treated the 20-log objective as satisfied if any part of the house existed, even without 20 logs currently held. Moved to Open questions: The progression owner must decide whether partial construction is sufficient evidence to bypass this objective. Until settled, partial construction alone is not accepted as proof that 20 logs were gathered.

5. Definition of night: The agent selected ticks 12542–23459 as its nighttime window and ignored weather. Moved to Open questions: The team must decide whether home-job selection depends on a fixed time window, Minecraft's sleep eligibility, or weather. The interim assumption is that weather alone does not activate nighttime home jobs.

6. Sleep refusal: The agent retained Minecraft's refusal message and continued retrying without a specified retry interval or limit. Moved to Open questions: The team must decide how frequently to retry and how to handle different refusal reasons. A refused attempt must not count as successful sleep.

7. Material failures: The agent introduced a RetryReporter interface to pass failures to another component. Deliberately left open: Internal interface names and organization may vary. The observable requirement is that failure leaves the objective incomplete and is reported to the retry/replanning process.

8. Action-validation limits: The agent invented gather and craft quantity limits, coordinate limits, and identifier-format restrictions. Fixed FR-4 (A): Validation must follow the project's documented supported-action schema and argument limits. The standalone implementation's invented limits are not accepted automatically.

9. Home-storage format: The agent selected a separate properties file for each world and dimension and used atomic saving. Deliberately left open: The storage format is an implementation choice, provided it preserves the required world/dimension association and satisfies restoration and error-handling requirements.

10. Home-file errors: The agent treated missing data as an unknown home without a warning, and malformed data as an unknown home with a warning. Fixed FR-10 (A): Added explicit missing-file and malformed-file behavior.

Requirements revised after the agent check

ID	Requirement	Priority	Traces to	Verified by
FR-4 (A)	Before executing a model-selected action, the system shall validate its type and arguments against the project's supported-action schema and documented limits. Unsupported types, missing required arguments, and out-of-range values shall be rejected without dispatching a game command.	Must	U-1, U-3	Submit valid, unsupported-type, missing-argument, and boundary-value fixtures; verify that only valid actions are dispatched.
FR-6 (A)	The system shall require every wall and roof barrier specified by the starter-house blueprint before declaring the shell complete. If a required cell is unloaded, the current inspection shall report the structure as unknown and shall not establish new completion.	Must	U-2	Test a complete shell, a shell with one missing barrier, and a shell with an unloaded required cell. Only the complete, fully loaded shell passes.
FR-10 (A)	The system shall save home coordinates for the current world and dimension and restore them on rejoin. A missing or malformed home file shall leave the home unknown without terminating the client. A malformed file shall additionally produce a warning.	Should	U-2, U-4	Compare coordinates before and after restart; test missing and malformed files; verify continued client operation and a warning for malformed data.

Verification limitations

The agent reported that the main sources compiled through Gradle before a final edit, which it checked separately using javac. The Gradle test task did not complete because JUnit dependencies could not be downloaded. The agent reported 25 passing cases using a homemade test substitute; this does not establish that the actual JUnit suite passed. No behavior was tested inside Minecraft, and the Fabric/Baritone adapters were not implemented.

Claude Opus 5.5 was used to explain our codebase so I understood the features I was specifying, and to review my draft for ambiguous or untestable requirements. I wrote the requirements myself and revised them based on its review.